Könyvespolc API

Bemutató REST API nulláról · Express 5 · egyetlen fájl · az adatok a memóriában

Ebben a feladatban egy egyszerű REST API-t készítesz, amely könyveket tart nyilván. Az adatokat nem fájlban és nem adatbázisban tárolja, hanem egy JavaScript-objektumban, a program memóriájában. Minden kódrészlet után ott van, mit csinál, és miért kell.

1. Mit készítünk?

Végpont	Mit tesz?	Válasz
`GET /api/konyvek`	az összes könyv; `?szerzo=` paraméterrel csak egy szerzőé	200
`GET /api/konyvek/:id`	egyetlen könyv	200 · 404
`POST /api/konyvek`	új könyv felvétele	201 · 400
`PUT /api/konyvek/:id`	egy könyv teljes cseréje	200 · 400 · 404
`PATCH /api/konyvek/:id`	egy könyv részleges módosítása	200 · 404
`DELETE /api/konyvek/:id`	egy könyv törlése	204 · 404

Egy könyv öt mezőből áll:

Mező	Típus	Jelentés
`id`	`szám`	egyedi azonosító — a szerver adja
`cim`	`szöveg`	a könyv címe, kötelező
`szerzo`	`szöveg`	a szerző neve, kötelező
`ev`	`szám vagy null`	a megjelenés éve
`olvasott`	`logikai`	elolvastad-e már

2. A projekt létrehozása

2.1 A mappa

Hozz létre egy **konyvespolc-api** mappát, nyisd meg a VS Code-ban, és nyiss benne egy terminált.

2.2 A package.json

```
npm init -y
```

Az `npm init` létrehozza a `package.json` fájlt. Ez írja le a projektet: a nevét, a futtatási parancsait, és azt, milyen csomagokra van szüksége. A `-y` annyit tesz: „ne kérdezz semmit, használd az alapértelmezéseket”.

Nyisd meg a fájlt, és írd át benne ezt a három részt:

Mező	Új érték	Miért?
`"type"`	`"module"`	az `npm init` `"commonjs"`-t írt ide. Így az `import` kulcsszó hibát adna: `Cannot use import statement outside a module`
`"description"`	`"Bemutató REST API: könyvespolc"`	üresen hagyta; írd bele, mit csinál a projekt
`"scripts"`	két saját parancs	lásd lent

A `scripts` szakasz `test` sorát cseréld le erre a kettőre:

```
"start": "node index.js",
"dev": "node --watch index.js"
```

- `start` — elindítja a programot. Ez különleges név, ezért elég annyi: `npm start`.
- `dev` — ugyanaz, de a `--watch` miatt minden mentéskor magától újraindul. A saját nevű parancsokat így kell indítani: `npm run dev`. Fejlesztés közben ezt használjuk.

2.3 Az Express telepítése

```
npm install express
```

Az Express a Node.js legelterjedtebb webes keretrendszere: a kérések fogadását és a válaszok küldését egyszerűsíti le. A parancs kimenete:

```
added 65 packages, and audited 66 packages in 3s
```

- Létrejön a `node_modules` mappa. Ebben van az Express, és az a 64 csomag, amelyre neki szüksége van.
- Létrejön a `package-lock.json`. Ez rögzíti, pontosan melyik verzió települt.
- A `package.json` kap egy `dependencies` szakaszt: `"express": "^5.3.0"`. A `^` azt jelenti: az 5-ös főverzión belül újabb is települhet, 6-os nem.

2.4 A .gitignore

Ha Gitet használsz, hozz létre egy `.gitignore` fájlt egyetlen sorral: `node_modules/`. Ez a mappa nagy, és az `npm install` bármikor újra előállítja a `package.json` és a `package-lock.json` alapján.

3. Az első futó szerver

Hozd létre az **index.js** fájlt, és írd bele:

```
import express from "express";

const app = express();
const PORT = 3000;
```

- `import express from "express";` — importáld az Express csomagot. A Node a `node_modules` mappában keresi meg.
- `const app = express();` — az `express` egy függvény; meghívva elkészíti az alkalmazást. Ehhez az `app` objektumhoz fűzzük majd a végpontokat.

- `const PORT = 3000;` — a port, amelyen a szerver a kéréseket várja. Egy portot egyszerre csak egy program foglalhat.

A fájl **végére** jön az indítás. Ez végig az utolsó rész marad: minden, amit később írsz, elé kerül.

```
app.listen(PORT, (hiba) => {
  if (hiba) throw hiba;
  console.log(`A szerver fut: http://localhost:${PORT}`);
});
```

- `app.listen(PORT, …)` — elindítja a szervert a megadott porton. A második paraméter egy callback: akkor fut le, amikor a szerver elindult — vagy amikor nem sikerült elindulnia.
- `if (hiba) throw hiba;` — ha a port foglalt, a callback a hibát kapja meg. Továbbdobva a program leáll, és kiírja az okát: `EADDRINUSE`. Enélkül azt írná ki, hogy „A szerver fut” — pedig nem fut.
- `console.log(…)` — kiírja a címet. A backtick közé írt szövegben a `${PORT}` helyére a változó értéke kerül.

Indítsd el:

```
npm run dev
```

A terminálon:

```
> konyvespolc-api@1.0.0 dev
> node --watch index.js

A szerver fut: http://localhost:3000
```

Nyisd meg a böngészőben a **http://localhost:3000** címet. Ezt kapod:

```
Cannot GET /
```

Ez nem hiba. A szerver fut, és válaszolt — csak még nincs egyetlen végpontja sem. Az Express ilyenkor egy 404-es HTML-oldallal felel.

A szerver addig fut, amíg le nem állítod: **Ctrl + C** a terminálban. Mentéskor nem kell újraindítanod — a `--watch` megteszi: a terminálban megjelenik a `Restarting 'index.js'` sor, és a szerver újra elindul.

4. Az adatok

4.1 A kérés törzsének feldolgozása

Közvetlenül a `const PORT = 3000;` sor után:

```
app.use(express.json());
```

- `app.use(…)` — egy olyan függvényt fűz a szerverhez, amely **minden** kérésnél lefut, mielőtt a kérés a végponthoz érne. Az ilyen függvény neve: middleware.
- `express.json()` — ez a middleware a JSON-formátumú kéréstörzset JavaScript-objektummá alakítja, és a `req.body`-ba teszi. Enélkül a `req.body` értéke `undefined` lenne, és a POST, a PUT és a PATCH nem kapná meg az adatokat.

4.2 A könyvek

Utána jöjjenek az adatok:

```
const adatok = {
  kovetkezoId: 16,
  konyvek: [
    { id: 1, cim: "A Pál utcai fiúk", szerzo: "Molnár Ferenc", ev: 1906, olvasott: true },
    { id: 2, cim: "Egri csillagok", szerzo: "Gárdonyi Géza", ev: 1899, olvasott: false },
    { id: 3, cim: "A láthatatlan ember", szerzo: "Gárdonyi Géza", ev: 1902, olvasott: false },
    { id: 4, cim: "Az arany ember", szerzo: "Jókai Mór", ev: 1872, olvasott: false },
    { id: 5, cim: "A kőszívű ember fiai", szerzo: "Jókai Mór", ev: 1869, olvasott: true },
    { id: 6, cim: "Légy jó mindhalálig", szerzo: "Móricz Zsigmond", ev: 1920, olvasott: true },
    { id: 7, cim: "Rokonok", szerzo: "Móricz Zsigmond", ev: 1932, olvasott: false },
    { id: 8, cim: "Tüskevár", szerzo: "Fekete István", ev: 1957, olvasott: true },
    { id: 9, cim: "Vuk", szerzo: "Fekete István", ev: 1965, olvasott: true },
    { id: 10, cim: "Abigél", szerzo: "Szabó Magda", ev: 1970, olvasott: false },
    { id: 11, cim: "Az ajtó", szerzo: "Szabó Magda", ev: 1987, olvasott: false },
    { id: 12, cim: "Édes Anna", szerzo: "Kosztolányi Dezső", ev: 1926, olvasott: false },
    { id: 13, cim: "Pacsirta", szerzo: "Kosztolányi Dezső", ev: 1924, olvasott: false },
    { id: 14, cim: "Toldi", szerzo: "Arany János", ev: 1847, olvasott: true },
    { id: 15, cim: "Utas és holdvilág", szerzo: "Szerb Antal", ev: 1937, olvasott: false },
  ],
};
```

- `const adatok = { … }` — egyetlen objektum, ez az „adatbázisunk”. Két mezője van.
- `kovetkezoId` — a következő új könyv azonosítója. Minden felvételnél eggyel nő.
- `konyvek` — a könyvek tömbje. Minden eleme egy objektum az 1. pont öt mezőjével.

Tizenöt könyv, több szerzőtől kettő-kettő, vegyesen olvasott és olvasatlan: így lesz mit szűrni és összesíteni — újraindítás után is.

Miért kell külön `kovetkezoId`, és miért nem elég a `konyvek.length + 1`? Mert törlés után a hossz csökken. A 15 könyvből az 1-est törölve 14 marad, és a következő új könyv a **15**-ös azonosítót kapná — pedig olyan már van. A külön számláló csak nő, így egy azonosító soha nem ismétlődik.

Az adatok a **memóriában** vannak. Minden újraindításkor visszaáll a fenti 15 könyv — és a `--watch` minden mentéskor újraindít. Ha felveszel egy könyvet, majd elmented az `index.js`-t, a könyv eltűnik. Egy bemutatóhoz ez elég; ha az adatoknak meg kell maradniuk, fájlba vagy adatbázisba kell írni őket.

5. Olvasás

A végpontok az `adatok` és az `app.listen(…)` közé kerülnek, egymás alá.

5.1 Az összes könyv

```
app.get("/api/konyvek", (req, res) => {
  res.json(adatok.konyvek);
});
```

- `app.get(útvonal, függvény)` — ha **GET** kérés érkezik erre az útvonalra, lefut a függvény. Ez a függvény a végpont kezelője.
- `req` — a beérkező kérés: innen olvassuk ki, mit kért a kliens.
- `res` — a válasz: ezen keresztül küldünk vissza adatot.

- `res.json(…)` — JSON-ná alakítja az adatot, beállítja a `Content-Type: application/json` fejlécet, és elküldi **200**-as kóddal.

Mentsd el, és nyisd meg a **http://localhost:3000/api/konyvek** címet: megjelenik a 15 könyv.

5.2 Szűrés szerző szerint

A kliens a cím végén, `?` után adhat meg paramétert: `/api/konyvek?szerzo=gárdonyi`. Bővítsd ki a kezelőt: a `res.json(adatok.konyvek);` sor **elé** írd ezt:

```
  const { szerzo } = req.query;

  if (szerzo) {
    const keresett = szerzo.toLowerCase();
    const talalatok = adatok.konyvek.filter((k) => k.szerzo.toLowerCase().includes(keresett));
    return res.json(talalatok);
  }
```

- `req.query` — a `?` utáni paraméterek, objektumként: `{ szerzo: "gárdonyi" }`. Az értékük mindig **szöveg**.
- `const { szerzo } = req.query;` — kiolvassa a `szerzo` mezőt egy ugyanilyen nevű változóba. Ha nincs ilyen paraméter, az értéke `undefined`.
- `if (szerzo)` — csak akkor szűrünk, ha a kliens megadta a paramétert.
- `toLowerCase()` — mindkét oldalt kisbetűssé alakítjuk, így a `gárdonyi` és a `Gárdonyi` is talál.
- `filter(…)` — új tömböt ad azokkal az elemekkel, amelyekre a függvény igazat ad. Az `includes` azt nézi, benne van-e a keresett szöveg a szerző nevében — így elég a név egy része is.
- `return res.json(talalatok);` — a `return` miatt a függvény itt véget ér. Nélküle a futás továbbmenne a `res.json(adatok.konyvek)` sorra, és a szerver egy kérésre **kétszer** próbálna válaszolni.

A válaszok:

```
/api/konyvek?szerzo=gárdonyi   ->  2 könyv: Egri csillagok, A láthatatlan ember
/api/konyvek?szerzo=petőfi     ->  []   (üres tömb, 200-as kóddal)
```

Az üres találat nem hiba: a kérés rendben volt, csak nincs ilyen könyv.

5.3 Egy könyv azonosító szerint

```
app.get("/api/konyvek/:id", (req, res) => {
  const id = Number(req.params.id);
  const konyv = adatok.konyvek.find((k) => k.id === id);
```

- `/:id` — az útvonal változó része. A `/api/konyvek/2` kérésnél az `id` értéke `"2"`.
- `req.params.id` — innen olvasható ki. Mindig **szöveg**, ezért a `Number()` számmá alakítja. Enélkül a `"2" === 2` hamis lenne, és a létező könyvre is 404 jönne.
- `find(…)` — az első elemet adja vissza, amelyre a függvény igazat ad, vagy `undefined`-ot, ha nincs ilyen.

A kezelő második fele:

```
  if (!konyv) {
    return res.status(404).json({ hiba: "Nincs ilyen azonosítójú könyv." });
  }

  res.json(konyv);
});
```

- `if (!konyv)` — ha a `find` nem talált semmit.

- `res.status(404)` — beállítja a státuszkódot. A **404** azt jelenti: nincs ilyen erőforrás. A rá fűzött `.json(…)` küldi el a hibaüzenetet.
- `return` — a hibaválasz után ne fusson tovább a kezelő.

A válaszok:

```
/api/konyvek/2     ->  200  {"id":2,"cim":"Egri csillagok","szerzo":"Gárdonyi Géza",…}
/api/konyvek/99    ->  404  {"hiba":"Nincs ilyen azonosítójú könyv."}
/api/konyvek/abc   ->  404  – a Number("abc") eredménye NaN, ilyen azonosító nincs
```

6. Postman

A böngésző címsorából csak GET kérés küldhető. A többi metódushoz Postman kell.

1. Hozz létre egy gyűjteményt **Könyvespolc API** néven.
2. Vegyél fel egy környezetet, benne egyetlen változót: `host` = `http://localhost:3000/api`. Jobb fent válaszd is ki.
3. A kérések címe így `{{host}}/konyvek` alakú lesz.

A törzset küldő kéréseknél (POST, PUT, PATCH) a törzs a **Body → raw → JSON** fülre kerül. A Postman ilyenkor magától beállítja a `Content-Type: application/json` fejlécet — az `express.json()` csak az ilyen kérés törzsét dolgozza fel.

7. Írás

7.1 POST — új könyv

```
app.post("/api/konyvek", (req, res) => {
  const { cim, szerzo, ev, olvasott } = req.body;

  if (!cim || !szerzo) {
    return res.status(400).json({ hiba: "A cím és a szerző megadása kötelező." });
  }
```

- `app.post(…)` — ugyanaz az útvonal, mint a listáé, de **más metódus**: az Express a kettőt külön kezeli.
- `req.body` — a kérés törzse, amelyet az `express.json()` már objektummá alakított.
- `const { cim, szerzo, ev, olvasott } = req.body;` — csak ezt a négy mezőt olvassuk ki. Ami ezen kívül érkezik — például egy `id` —, azt figyelmen kívül hagyjuk.
- `!cim || !szerzo` — ha bármelyik hiányzik, vagy üres szöveg. A válasz **400**: a kliens hibás kérést küldött.

A kezelő folytatása:

```
  const uj = {
    id: adatok.kovetkezoId,
    cim,
    szerzo,
    ev: ev ?? null,
    olvasott: olvasott ?? false,
  };
```

- `id: adatok.kovetkezoId` — az azonosítót a szerver adja, nem a kliens.
- `cim,` — rövidített írásmód, ugyanaz, mint a `cim: cim`.
- `ev ?? null` — a `??` jobb oldala akkor lép életbe, ha a bal oldal `null` vagy `undefined`. Ha a kliens nem küldött évet, `null` lesz; ha küldött, az marad.

- `olvasott ?? false` — ha nem küldték, a könyv még olvasatlan.

A kezelő vége:

```
  adatok.kovetkezoId += 1;
  adatok.konyvek.push(uj);

  res.status(201).json(uj);
});
```

- `adatok.kovetkezoId += 1;` — a számláló eggyel nő, így a következő könyv új azonosítót kap.
- `push(uj)` — a tömb végére teszi az új könyvet.
- `res.status(201)` — a **201** azt jelenti: létrejött. A válaszban visszaküldjük a kész könyvet — benne az azonosítóval, amelyet a kliens nem tudhatott előre.

Postmanben: **POST** `{{host}}/konyvek`, a törzs:

```
{ "cim": "Az ember tragédiája", "szerzo": "Madách Imre", "ev": 1861 }
```

A válasz **201**:

```
{"id":16,"cim":"Az ember tragédiája","szerzo":"Madách Imre","ev":1861,"olvasott":false}
```

Az `olvasott` mezőt nem küldtük, ezért az alapértékét kapta.

> Miért nem másoljuk át egyszerűen az egész `req.body`-t (`{ id: …, ...req.body }`)? Mert akkor a kliens bármit beleírhatna. Egy `{"id": 1, "cim": "Trükk", "szerzo": "Valaki"}` törzs után **két** 1-es azonosítójú könyv lenne, és a `GET /api/konyvek/1` csak az elsőt adná vissza — az újat azonosító alapján soha többé nem lehetne elérni.

7.2 PUT — teljes csere

```
app.put("/api/konyvek/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = adatok.konyvek.findIndex((k) => k.id === id);

  if (index === -1) {
    return res.status(404).json({ hiba: "Nincs ilyen azonosítójú könyv." });
  }
```

- `findIndex(…)` — most nem az elemet keressük, hanem a **helyét** a tömbben, mert oda kell majd visszaírni az új változatot. Ha nincs találat, `-1`-et ad.

A folytatás:

```
  const { cim, szerzo, ev, olvasott } = req.body;

  if (!cim || !szerzo) {
    return res.status(400).json({ hiba: "A cím és a szerző megadása kötelező." });
  }
```

- Ugyanaz az ellenőrzés, mint a POST-nál. A PUT a könyv **teljes, új állapotát** kapja, ezért a kötelező mezőknek itt is meg kell lenniük.

A kezelő vége:

```
  const csere = {
    id,
    cim,
    szerzo,
    ev: ev ?? null,
    olvasott: olvasott ?? false,
  };

  adatok.konyvek[index] = csere;
  res.json(csere);
});
```

- `id,` — az azonosító az útvonalból jön, nem a törzsből: a csere nem változtathatja meg.
- `adatok.konyvek[index] = csere;` — a régi könyv helyére az új objektum kerül.
- Amit a kliens nem küldött el, az az **alapértékére** áll — ez a PUT lényege.

Postmanben: **PUT** `{{host}}/konyvek/2`, a törzs csak a cím és a szerző:

```
{ "cim": "Egri csillagok", "szerzo": "Gárdonyi Géza" }
```

A válasz: az `ev` eddig 1899 volt, most `null` lett.

```
{"id":2,"cim":"Egri csillagok","szerzo":"Gárdonyi Géza","ev":null,"olvasott":false}
```

7.3 PATCH — részleges módosítás

```
app.patch("/api/konyvek/:id", (req, res) => {
  const id = Number(req.params.id);
  const konyv = adatok.konyvek.find((k) => k.id === id);

  if (!konyv) {
    return res.status(404).json({ hiba: "Nincs ilyen azonosítójú könyv." });
  }
```

- Itt `find`-ot használunk, nem `findIndex`-et. A `find` által visszaadott objektum **ugyanaz**, amelyik a tömbben van, nem egy másolata. Ha módosítjuk, a tömbben lévő könyv módosul.

A kezelő vége:

```
  const { cim, szerzo, ev, olvasott } = req.body;

  if (cim !== undefined) konyv.cim = cim;
  if (szerzo !== undefined) konyv.szerzo = szerzo;
  if (ev !== undefined) konyv.ev = ev;
  if (olvasott !== undefined) konyv.olvasott = olvasott;

  res.json(konyv);
});
```

- Csak azt írjuk felül, ami **megérkezett**. Ami nincs a törzsben, az `undefined`, és változatlan marad.
- Miért `!== undefined`, és miért nem egyszerűen `if (olvasott)`? Mert a `false` is érték. `if (olvasott)` mellett az `{"olvasott": false}` törzs **semmit nem változtatna** — a válasz 200 lenne, a könyv pedig olvasott maradna.

Postmanben: **PATCH** `{{host}}/konyvek/3`, a törzs:

```
{ "olvasott": true }
```

A válasz: csak az `olvasott` változott, minden más maradt.

```
{"id":3,"cim":"A láthatatlan ember","szerzo":"Gárdonyi Géza","ev":1902,"olvasott":true}
```

	PUT	PATCH
mit küld a kliens?	a könyv teljes, új állapotát	csak azt, ami változik
a nem küldött mezők…	alapértékre állnak	változatlanok maradnak
kötelező a cím és a szerző?	igen	nem
mire jó?	egy szerkesztőűrlap mentésére	az „elolvastam” jelölésre

7.4 DELETE — törlés

```
app.delete("/api/konyvek/:id", (req, res) => {
  const id = Number(req.params.id);
  const index = adatok.konyvek.findIndex((k) => k.id === id);

  if (index === -1) {
    return res.status(404).json({ hiba: "Nincs ilyen azonosítójú könyv." });
  }

  adatok.konyvek.splice(index, 1);
  res.status(204).end();
});
```

Az eleje ugyanaz, mint a PUT-é. Új a vége:

- `splice(index, 1)` — az `index` helyről **egy** elemet kivesz a tömbből. A mögötte lévők előrecsúsznak.
- `res.status(204).end()` — a **204** azt jelenti: sikerült, de nincs mit visszaküldeni. Az ilyen válasznak nincs törzse, ezért `end()`, és nem `json()`.

Postmanben: **DELETE** `{{host}}/konyvek/1`. A válasz **204**, üres törzzsel. Ugyanez másodszor: **404** — az 1-es könyv már nincs meg.

8. A kész fájl sorrendje

Az `index.js` fentről lefelé:

1. az Express importálása, az `app` és a `PORT`
2. `app.use(express.json())`
3. az `adatok` objektum
4. `GET /api/konyvek` és `GET /api/konyvek/:id`
5. `POST`, `PUT`, `PATCH`, `DELETE`
6. `app.listen(…)` — mindig a legvégén

9. Sorvezető

Amit használsz	Mit csinál?
`express()`	létrehozza az alkalmazást
`app.use(fv)`	a függvényt minden kérés elé fűzi
`app.get / post / put / patch / delete`	egy metódust és egy útvonalat köt egy kezelőhöz
`app.listen(port, fv)`	elindítja a szervert; a callback hibát is kaphat
`req.params`	az útvonal változó részei (`:id`) — szövegként
`req.query`	a `?` utáni paraméterek — szövegként
`req.body`	a kérés törzse — az `express.json()` után
`res.json(adat)`	JSON-választ küld (alapból 200)
`res.status(kód)`	beállítja a státuszkódot
`res.status(204).end()`	üres válasz, törzs nélkül
`tomb.find(fv)`	az első illeszkedő elem, vagy `undefined`
`tomb.findIndex(fv)`	az első illeszkedő elem helye, vagy `-1`
`tomb.filter(fv)`	az illeszkedő elemek, új tömbben
`tomb.splice(index, 1)`	egy elem kivétele a tömbből
`a ?? b`	`b`, ha a értéke `null` vagy `undefined`

10. Ha nem működik

Tünet	Mi lehet a baj?
`Cannot use import statement outside a module`	a `package.json`-ben maradt a `"type": "commonjs"`
`EADDRINUSE` induláskor	egy másik terminálban már fut a szerver — ott állítsd le
`Cannot GET /api/konyvek`	elírt útvonal, vagy nem mentetted el a fájlt
500, `Cannot destructure property 'cim' of 'req.body'`	hiányzik az `express.json()`, vagy a Postmanben nem **raw** → **JSON** a törzs (a **Text** törzs is ezt okozza)
a létező könyvre is 404	kimaradt a `Number()`
400, pedig küldtél címet	elgépelt mezőnév: `"Cim"` vagy `"cím"` nem ugyanaz, mint `"cim"`
eltűnt a felvett könyv	újraindult a szerver — például mert mentettél. Az adatok a memóriában vannak

11. Próbáld ki

Mindegyiket egy működő változaton érdemes kipróbálni, utána állítsd vissza.

1. Vedd ki a `Number()` hívást az egy könyvet lekérő végpontból. A `/api/konyvek/2` **404**-et ad, pedig a 2-es könyv létezik.
2. Vedd ki az `app.use(express.json());` sort, és küldj egy POST kérést. **500**-as HTML-oldalt kapsz: `Cannot destructure property 'cim' of 'req.body' as it is undefined.`
3. A PATCH-ben írd át az utolsó feltételt `if (olvasott)`-ra, és küldd el: `{"olvasott": false}`. A válasz 200, de a könyv **olvasott marad**.
4. Vegyél fel egy könyvet, aztán mentsd el az `index.js`-t módosítás nélkül, és kérd le a listát. Az új könyv eltűnt.
5. Indítsd el a szervert egy második terminálban is. A második `EADDRINUSE` hibával leáll.
6. A szűrésnél vedd ki a `return` szót, és kérd le a `?szerzo=gárdonyi` címet. A kliens még a jó választ kapja, de a terminálon megjelenik: `Cannot set headers after they are sent to the client` — a szerver másodszor is válaszolni próbált.

12. Önálló feladatok

Ezekhez nincs kész kód — az eddigiek alapján meg tudod írni őket.

1. **Statisztika.** Készíts egy `GET /api/statisztika` végpontot, amely ezt adja vissza: hány könyv van összesen, hányat olvastál el, és hány van még hátra.
2. **Az év ellenőrzése.** A POST és a PUT adjon **400**-as választ, ha az `ev` megérkezett, de nem egész szám. (Tipp: `Number.isInteger`.)
3. **Csak az elolvasottak.** Készíts egy `GET /api/konyvek/olvasott` végpontot, amely csak az elolvasott könyveket adja vissza. Vedd fel először a `/api/konyvek/:id` végpont **után**, és próbáld ki. Mit kapsz, és miért? Hová kell tenni, hogy működjön?

13. Ellenőrzőlista

1. A `package.json`-ben `"type": "module"` áll, és az `npm run dev` mentéskor újraindul.
2. A `/api/konyvek` 15 könyvet ad, a `?szerzo=gárdonyi` kettőt.
3. A `/api/konyvek/2` **200**-at ad, a `/api/konyvek/99` **404**-et.
4. Az új könyv **201**-et kap, a 16-os azonosítót és `"olvasott": false` értéket; cím nélkül **400** a válasz.
5. A PUT a nem küldött mezőket alapértékre állítja, a PATCH nem.
6. A PATCH az `{"olvasott": false}` törzzsel is működik.
7. A DELETE **204**-et ad törzs nélkül, másodszor **404**-et.
8. A kliens nem tud azonosítót megadni: a törzsben küldött `id`-t a szerver figyelmen kívül hagyja.